- **History of AI & ML**
  - **1950s:** Very early on Neural Networks & Machine Learning mentioned as a concept
  - **1980s: Symbolic AI & Search** → **Learning from Data** (shift to learning patterns from data)
  - **From the 2000s:** Deep Learning Revolution → Transfomers, CNNs, & advanced NNs
- Different ML paradigms we'll go over
  - *Supervised (Regression & Classification), UnSupervised (Clustering, grouping etc.) , Self-Supervised Learning*
- *"Features"* → Up to us, what we do with that

- What is a "good" fit in Linear Regression?
  → With a **Loss/Cost Function**
  - **Aim:** Quantifying how good the *trained* model fits a specific dataset (emphasis on ***after learning***)
- What is **Loss**?
  - Tells us how well a trained model fits a specific dataset & exhibits the patterns of this data
  - **Alternatively**: Quantifies how much our model's predicitions deviate from the data we train on

💡 Where does squared loss come from?

We can derive least squares based on MLE:

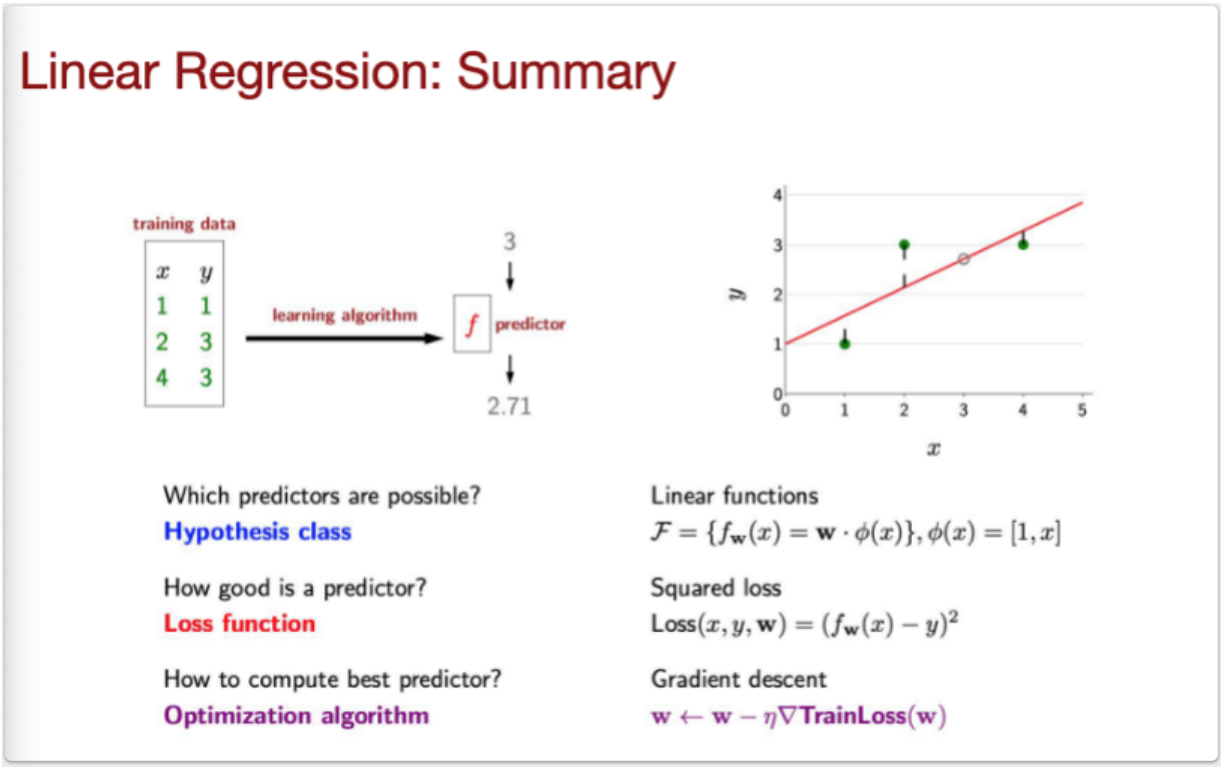
1. We assume for linear regression: $y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)}$
2. We assume the noise $\epsilon^{(i)}$ is normal that is independent and has a zero mean with variance σ^2. Therefore,

$$p(y^{(i)} \mid x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

3. Finding the argument θ that now maximized this function, leaves us with the best set of parameters θ to observe the data Y given X
4. Calculating this, we get the minimized squared error

- 💡 What is a closed form?
  - Formula that can be calculated with a finite number of operations - no infinite sums or iterations!
  - They are preferred because they can be computed efficiently & directly

- 💡 Using averages and normalizing makes floating point errors unlikely in computing (we like small numbers)



Linear Regression - Summary

- **Model Loss** → We only have generally 2 high-level categories of loss that we bother about when training a model
  - **Training Loss** → The quantification of how well our model fits our training data (or the deviation from the training data)
  - **Generalization Error** → How well do we fit the underlying distribution of the data (all possible data)?
    - A **perfect model** would try to get the generalization error to a minimum (based on the limited data we have)
    - As we usually don't have access to **all data**, we can use the next-best thing, which is **test data** (unseen data that we have but don't use to train)
      - The test-data is the best proxy for the **generalization error** we have!

Model Assumptions

When we are trying to train a **Machine Learning model**, we make assumptions about the data we are trying to predict and learn from:

- **Hypothesis Class** (H/F): This is the set of **all possible functions** that our learning algorithm can choose from
  - This is based on our ***assumption*** on how we think the datapoints (features and targets) are related to each other.
  - This can be e.g. a linear function, or a sine wave or a parabola.
  - Our model's complexity is determined by the size & flexibility by that Hypothesis class
- **Predictor** (f): This is the final function that our learning algorithm chooses after being trained on the data. It's the "best" candidate

> **How do these assumptions apply to Neural Networks?**
>
> Neural Networks are special, as they allow for much more flexibility and have in itself a vast hypothesis class, including all complex, non-linear functions.
> → This specifically is the strength of **Neural Networks**!

But still, we do some assumptions, like our loss function or the different activation functions and the higher-level design of it (complexity, activation functions, amount of neurons in the layers)

Model Complexity

The model complexity very much depends on the chosen Hypothesis Class of the model we are trying to train, and its inherent **flexibility in terms of hyperparameters** like *features*

Does the size of training data influence complexity?

No, it doesn't. More training data is usually always better, as it allows us to build a more representative dataset that is closer to the true distribution f .

As we add more data:

- The Training Error starts at zero, and then rises
- The Generalization Error starts high (due to high bias, low variance) but then falls!

Model Design & Performance

To better describe our model's performance, we can look at what makes up on our model's error on a high-level:

$$\text{Expected Prediction Error(x)} = \text{Noise} + \text{Bias}^2 + \text{Variance}$$

Noise, **Bias** & **Variance** are describing how

- difference in outputs inherent to the stochastic nature of the data (always there)
- well on **average our models** predict the true predictor f of the underlying distribution of our data (**Bias**)
- and how sensitive our model is to **noise** and **individual datasets** (**Variance**)

Bias & Variance

Variance → A property of the model - It describes the model's sensitivity to ***noise***

- What is **varying** are the different change in predictions when training this models with different datasets (that all have different random noise terms)

Bias → The difference between the *average* prediction of our model and the true function $f(x)$ of the population (which we probably will never know)

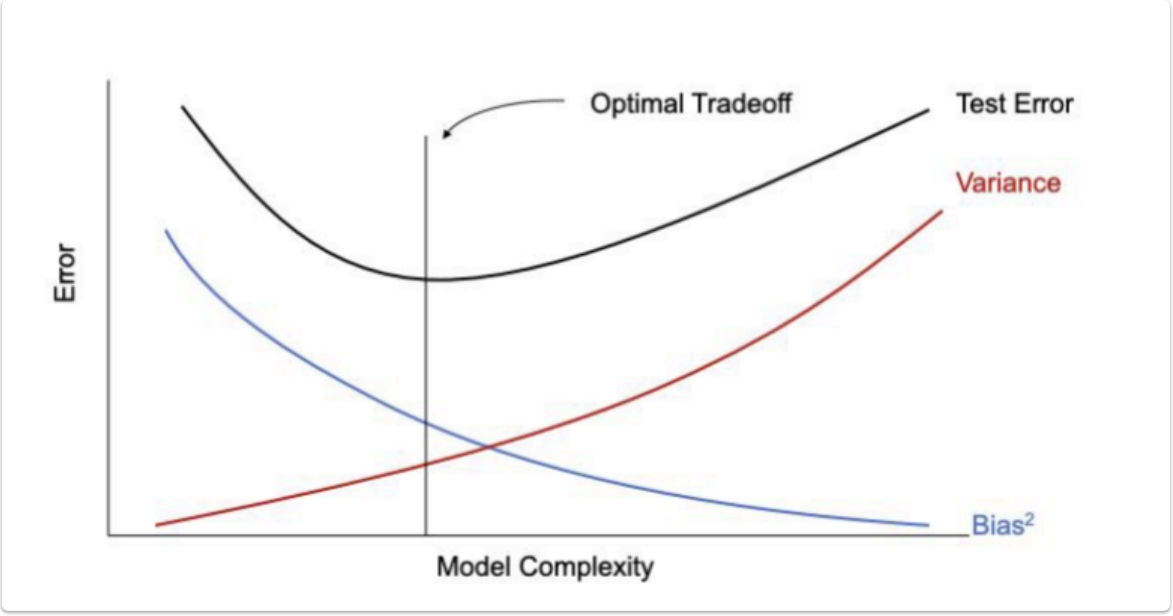
- Assuming our sample D is representative of the whole population, we use it as the best available proxy for that population to then calculate the bias
  - By creating *multiple smaller training samples* from D, we can simulate this process of calculating the **average prediction**

Insight

Variance models **how much noise we learned**, and Bias how bad **we learned the signal**

Bias vs. Variance

The ***variance-bias tradeoff*** can be imagined like a slider that we need to set for a vertical landscape, where the bottom is the underlying, smooth terrain (signal) and on top there is a **rough, spiky layer** which represents the layer. Depending on our model complexity (e.g. parameters we use to approximate the true function f)



There are three different cases that can happen

- A **high-variance** (low-bias) model is then so **flexible** that it contorts itself to every single data point perfectly (but also learns the **noise** ε next to the true signal $f(x)$
  - It's predictions across different training datasets are varying a lot more (as it learned signal *as well as* noise) - but on average across models with different datasets, it will have modeled the signal pretty well (when averaging noise cancels out) (→ *low-bias*)
  - ! High-variance models are in essence Overfitting!
- A **low-variance** (high-bias) model is not flexible enough to fit the noise and tends to find a general trend and ignore noise
  - Its predictions across different datasets *are more stable* but there can be **high bias** in case the model is too simple & doesn't get the relationship right (as it doesn't capture the whole **signal)**

Bias & Variance move antithetically to each other, as usually a model with lower variance tends to simplify more of the underlying assumptions of the population, and hence systematically be higher off the actual prediction $f(x)$

> **How to balance?**
>
> The art is to find the sweet trade-off spot, where **bias & variance are both low** - because if our model is too simple, it won't capure the signal fully, but if it is too complex then it also models the specific noise of our training set, not generalizing too well

Model Training

How do we find **optimal parameters** for a model that we build?

- Either we have a **closed-form solution** for parameters, that we can use
- or we use Gradient Descent, where we can optimize the Loss Function to find iteratively critical points of the function